

# Big Data Methods

Leon Kriesmair, Victoria Vorre & William von Tucher

May 7, 2025

## 1 Introduction

### 1.1 Problem Statement

In 2024 alone, it is estimated that wildfires caused around 250 to 270 billion USD of damage just in the US.<sup>1</sup> This figure includes the costs of damage, loss of life, healthcare, business disruptions, and other economic impacts associated with the fires. Given the increasing wrath and the number of wildfires each year, institutions and fire fighters must be sure to manage their limited resources efficiently. Confronted with a multitude of newly discovered fires it has to be decided on which of these to focus more resources on and on which less, a kind of fire triage.

### 1.2 Approach

Using a dataset of over 50,000 wildfires in the US, we test a variety of machine learning algorithms to research the extent to which these models can help classify the size of wildfires upon discovery. The data set includes weather information, a measure of remoteness, vegetational data, location, and temporal information.

The original data set holds the fire size in acres and the fire size class from A-G according to the scale of the National Wildfire Coordinating Group, where A is the smallest (0-0.25 acres) and G the largest (5000-9999 acres). This data set only holds fires of category B and larger, with a large imbalance towards category B fires. To efficiently triaging the fires upon discovery, it is more important to know whether the fire is potentially catastrophic or not, rather than the exact class it is in. For this reason and the present class imbalance, we propose a binary classification problem of "Not Catastrophic" (Class B: 0.26-9.9 acres) and "Catastrophic" (Higher than Class B: 10.0-9999 acres). This results in 36,522 "Not Catastrophic" wildfires ( 66%) and 18,845 "Catastrophic" wildfires ( 34%).

---

<sup>1</sup>LA Times Article

## 2 Data

### 2.1 Data Cleaning and Transformation

First, we remove the features that would either have no explaining value or that are not available upon discovery of a fire. That includes information on the containment such as time or date, or irrelevant features such as the name of a fire. We keep the following variables:

1. Fire Size Class (B-G)
2. Latitude, Longitude
3. Month of Discovery
4. Weather data (30, 15, and 7 days prior to discovery) at location of the fire
  - Temperature in Celcius
  - Humidity in %
  - Wind Speed in m/s
  - Precipitation in mm (cumulative over days)
5. Remoteness (non-dimensional distance to closest city)
6. Vegetation (Codes from 0-28), in our data set the following are present:
  - 0. Unknown
  - 4. Temperate Evergreen Needleleaf Forest
  - 9. C3 Grassland/Steppe
  - 12. Open Shrubland
  - 14. Desert
  - 15. Polar Desert/Rock/Ice
  - 16. Secondary Tropical Evergreen Broadleaf Forest

None of the 55,367 observations include any directly obvious missing values or NAs. Upon further inspection we find that observations where no weather data at all is recorded hold values of -1.0 in all weather columns. Thereby we already lose 15,297 observations ( 28%). For humidity, wind speeds, and temperature we find 38,545 individual zeros across these nine (one 7, 15, 30 day triplet for each variable) variables. We interpret those also as missing values, since exact zeros are either impossible (humidity/wind) or highly unlikely (temperature). We iterate over every triplet for each row. If any triplet holds three zeros, the row is dropped. If at least one value is present, the missing values are interpolated. If only one exists, it is simply used for the other two. If the 15 day prior feature is missing, we take a weighted average of the 30-day and

7-day value, weighted according to the difference in days. If an edge case is missing, we built a trend from the existing two and add that to get our new value. Here we also account for the difference in days and either halve the trend (if 7 day value is missing) or double it (if 30 day value is missing). We finally end up with 37,208 observations ( 67% of original dataset).

We convert the month of discovery (integer from 1 to 12) to a cyclical feature by mapping it to the unit circle. This results in a sine and a cosine feature, which account for January and December being next to each other.

Whether vegetation is used one-hot-encoded or as a single categorical feature depends on the model being used. Using the one-hot encoded version, we end up with a dataset of size 37208 x 26.

Finally, we split our dataset into 80% training and 20% testing data. Standardization is applied recursively during cross validation splits, refitting on the respective training data.

## 2.2 Descriptive Analysis

To better understand the distribution and scale of the features in the cleaned dataset, we compute basic descriptive statistics such as mean, standard deviation, median, and range for all continuous variables. Table 1 summarizes the key statistics for the main predictors and the outcome variable (catastrophic).

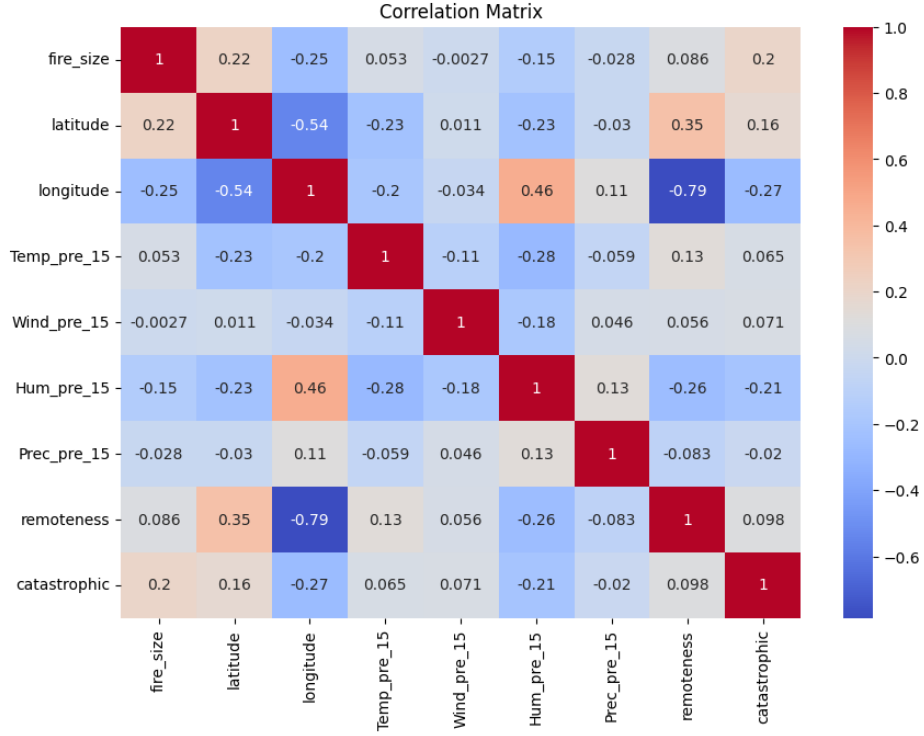
|              | mean    | std      | min     | median | max       |
|--------------|---------|----------|---------|--------|-----------|
| fire_size    | 2211.28 | 14890.17 | 0.51    | 4.00   | 538049.00 |
| latitude     | 36.48   | 6.81     | 17.96   | 34.70  | 69.05     |
| longitude    | -95.64  | 15.88    | -165.12 | -93.19 | -65.42    |
| Temp_pre_15  | 16.25   | 8.42     | -37.00  | 16.07  | 51.57     |
| Wind_pre_15  | 3.16    | 1.23     | 0.02    | 3.05   | 29.80     |
| Hum_pre_15   | 60.04   | 14.18    | 5.62    | 63.04  | 96.00     |
| Prec_pre_15  | 17.72   | 68.62    | 0.00    | 0.00   | 2527.00   |
| remoteness   | 0.24    | 0.14     | 0.00    | 0.21   | 1.00      |
| catastrophic | 0.35    | 0.48     | 0.00    | 0.00   | 1.00      |

**Table 1:** Descriptive Statistics of Selected Features

Several variables have a strong variety in their distributions. For instance, precipitation has a median of 0.0 mm but a maximum value exceeding 2500 mm, meaning that in most cases there is no rain, but in a few cases, rainfall is very high. In the same way, fire size has a median of 4.0, but some fires grow extremely large, reaching a size greater than 500,000 acres. In contrast, wind speed, temperature, and humidity show more symmetric distributions with reasonable ranges.

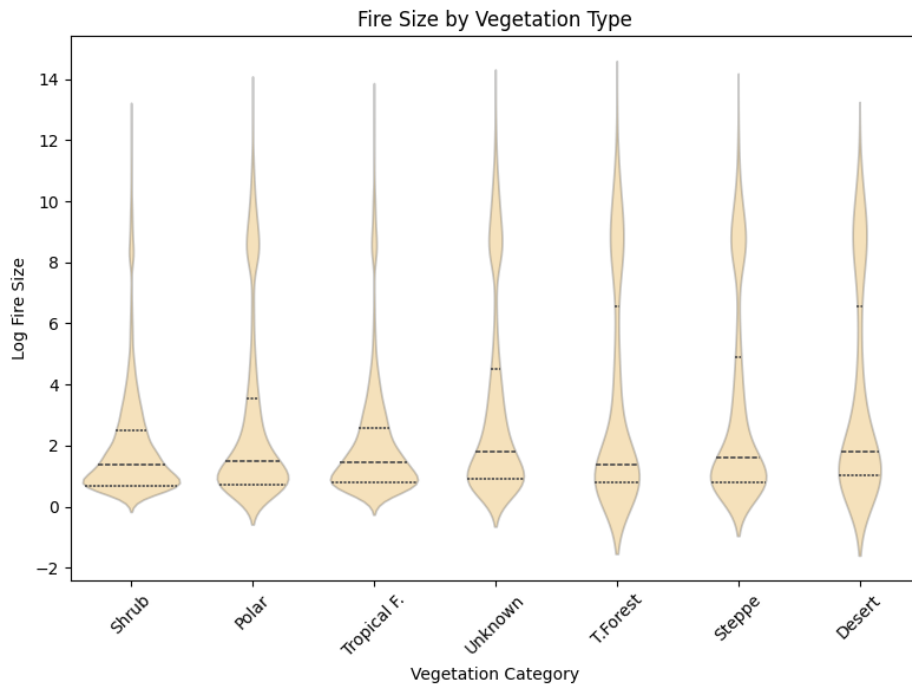
To further examine potential relationships between predictors and the outcome variable, we analyzed the correlation matrix, which can be found in Figure 1. In general, no single predictor shows a strong linear correlation with the target variable. The highest observed correlations

involve longitude, which shows a slight negative relationship with both fire\_size (-0.25) and catastrophic (-0.27), potentially suggesting that geographical location of the fire may indicate the fire outcome to some extent. Other predictors as remoteness (0.10) and humidity (-0.21) show weak but potentially relevant relationships. However, the absence of strong linear correlations suggests that catastrophic fires are driven by more complex, non-linear interactions between variables. Thus, justifying the following approaches of prediction.



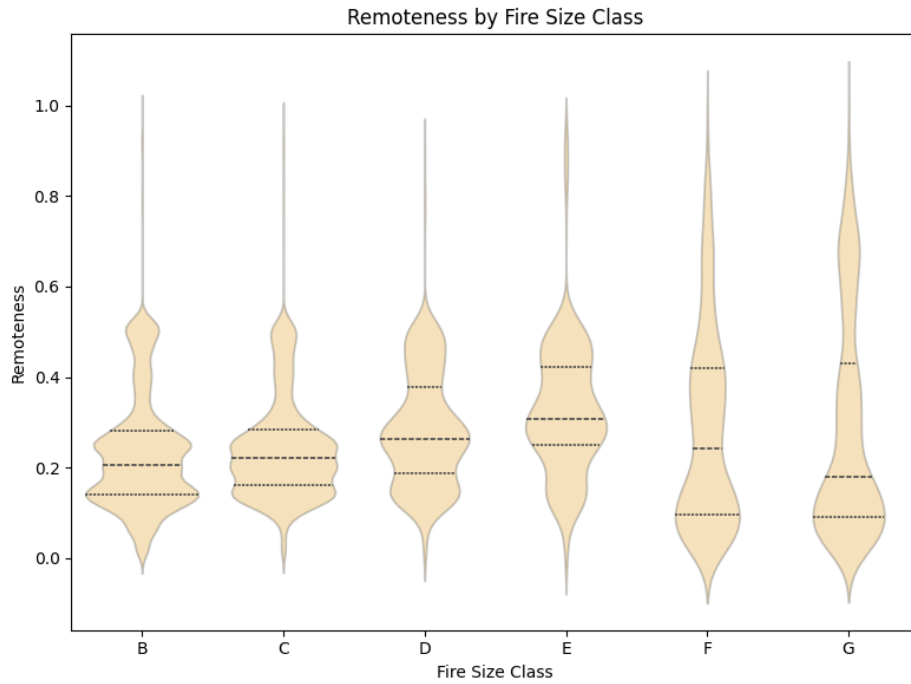
**Figure 1:** Correlation Matrix

While the correlation matrix provides a useful overview of linear relationships among continuous variables, it does not capture the role of categorical features such as vegetation. To assess the potential impact of vegetation, we examine the distribution of log-transformed fire sizes across vegetation types using a violin plot, which can be found in Figure 2. The distribution varies considerably between vegetation categories. Fires in polar deserts, tropical forests, and temperate forests tend to show longer upper tails, indicating that while most fires are small, some grow significantly larger. In contrast, vegetation types like shrublands and grasslands/steppes display a more compact distribution. Although the median fire size remains relatively consistent across most categories, the presence of thicker upper tails in certain vegetation types suggests that vegetation may influence the potential for large fires.

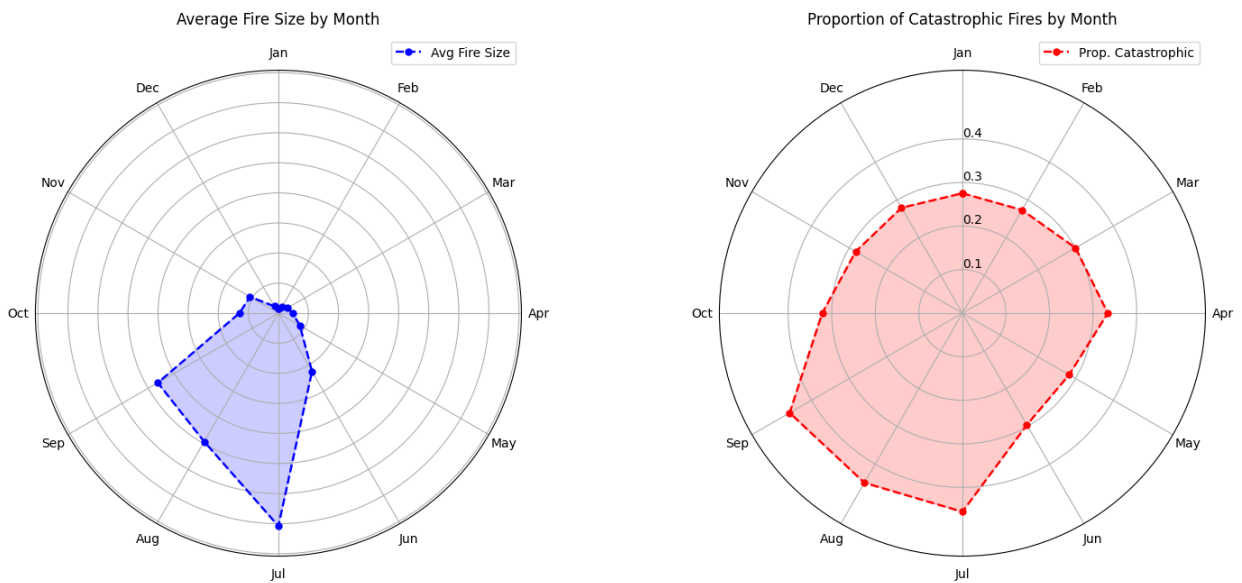


**Figure 2: Violin Plot: Vegetation**

In addition, we explore individual predictors that may play an important role in influencing fire size, even if their linear correlation is weak. One such variable is remoteness, which measures the non-dimensional distance to the closest city. The violin plot of remoteness can be found in Figure 3. Although the linear correlation between our target variable catastrophic and remoteness is low, visual analysis reveals a trend: larger fire size classes (E, F, G) tend to occur in more remote regions. The distribution of remoteness becomes wider and shifts upward for larger fires, suggesting that remote areas may contribute to the likelihood of fires growing out of control. Therefore, this relationship might carry significant predictive value.



**Figure 3:** Violin Plot: Remoteness



**Figure 4:** Temporal patterns: Size and Catastrophic fires pr. month

Furthermore, to analyze how fire severity changes throughout the year, we analyzed two temporal patterns using monthly polar plots. The plot on the left of Figure 4 shows the average fire size by month, while the plot on the right visualizes the proportion of catastrophic fires per month.

In both illustrations seasonal patterns are apparent. The proportion of catastrophic fires remains relatively constant throughout the months of October to June and increases dramatically in July, peaks until September. This suggests that fires that occur in the late summer are more likely to escalate to catastrophic levels.

A similar trend appears for the average fire size per month. The mean fire size rises dramatically

from June onward, reaching its peak in August, before declining again in the autumn months. This seasonal pattern between large fires and higher proportions of catastrophic outcomes indicates that seasonal factors, such as high temperature and dry vegetation, contribute to the escalation of fires.

## 3 Models

In the following, we describe the different models we used, the tuning process, and evaluate them using the default classification threshold of 0.5. We tune hyperparameters according to positive recall to try and avoid misclassifying potentially catastrophic fires as not being dangerous.

The last chapter of this section compares the models and highlights the role decision thresholds play in the presence of imbalanced classes.

### 3.1 Logistic Regression

#### 3.1.1 Introduction to the model

Logistic regression is a classification algorithm used for binary outcomes, making it well-suited for our project, where the goal is to predict whether a fire is catastrophic or not. When using logistic regression, we model the probability that a given input belongs to a particular class, using the logistic (sigmoid) function:  $P(catastrophic = 1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 latitude + \dots + \beta_n Vegetation)}}$ .

We will use the logistic regression model as our baseline model, since it is simple to interpret and fast to train.

#### 3.1.2 Final tuning/model

Since the dataset is clearly imbalanced, with most observations falling into class B, we decided to address this issue directly, by using a tuned logistic regression model with class weights. This gives us a fair and interpretable starting point to compare more complex models like Random Forests.

We then used GridSeachCV to tune the hyperparameters, with recall on the minority class as the scoring metric, and thus ensuring that the model is both well-calibrated and focused on the most important class: catastrophic fires. We chose to do the GridSearch even though logistic regression has fewer meaningful hyperparameters, so GridSearch may not add much benefit.

By default, logistic regression classifies an observation as positive (e.g., catastrophic fire) if the predicted probability is greater than 0.5. In our case, we used this standard threshold of 0.5, meaning the model predicts a fire to be catastrophic if it assigns a probability above 50 pct.

Later, when we compare our models, we will experiment with different thresholds to test the recall score.

| Parameter        | Values Tested                  | Best Value |
|------------------|--------------------------------|------------|
| logreg__C        | {0.001, 0.01, 0.1, 1, 10, 100} | 0.001      |
| logreg__penalty  | {L1, L2}                       | L1         |
| logreg__solver   | {liblinear, saga}              | liblinear  |
| logreg__max_iter | {500, 1000}                    | 500        |

**Table 2:** Parameter Grid for Logistic Regression (GridSearchCV)

The parameter grid used to do the GridSearch can be found in Table 2.

Logistic regression tries to find the best-fitting model by minimizing a loss function that measures prediction errors. To avoid overfitting, a penalty term called regularization is added. This keeps the model simpler by shrinking the coefficients. In our tuning, we adjusted the regularization strength using the C parameter, and tested both L1 (Lasso) and L2 (Ridge) penalties. These control how the model handles features and help to improve generalization. We also tested different solvers (liblinear, saga) that work well with these settings and imbalanced data. The hyperparameter C controls the strength of the regularization. A smaller value for C (e.g. C=0.01) leads to stronger regularization and a simpler model, while a greater value leads to weaker regularization and a more complex model.

The best parameters from the GridSearch are available in Table 2.

### 3.1.3 Model evaluation, interpretation and conclusion

| Metric / Feature                 | Overall | Catastrophic (Class 1) | Other (Class 0) |
|----------------------------------|---------|------------------------|-----------------|
| <b>Test Accuracy</b>             | 0.658   |                        |                 |
| <b>Recall</b>                    |         | 0.54                   | 0.72            |
| <b>Precision</b>                 |         | 0.51                   | 0.75            |
| <b>F1 Score</b>                  |         | 0.52                   | 0.73            |
| <b>CV Recall</b>                 |         | 0.531                  |                 |
| <b>Key Non-Zero Coefficients</b> |         |                        |                 |
| Longitude                        | -0.361  |                        |                 |
| Humidity_pre_15                  | -0.096  |                        |                 |
| Humidity_pre_7                   | -0.020  |                        |                 |

**Table 3:** Summary of Logistic Regression Results (Catastrophic = 1)

We used a tuned logistic regression model as a baseline classifier, optimized for recall on catastrophic fires, meaning that the model prioritizes detecting catastrophic fires. The overall accuracy of the model is moderate at 65.8 pct., while the recall score of catastrophic fires is 54 pct. This reflects a strategic trade-off: accepting some false positives to better catch true catastrophic fires, which can be appropriate in better predicting wildfire size, from a risk point of

view.

As the best regularization was chosen to be L1 via CV, it means that most coefficients are shrunk to zero, why only a few features had a measurable influence on predicting catastrophic fires.

However, in general we see, that low humidity increases the likelihood of the fire being catastrophic, which is also known driver of severe fire behavior from the real world.

To interpret the coefficient of "Longitude" we see that for every 1 unit increase in the variable, the log-odds of a fire being catastrophic increases by 0.361, holding all other variables constant. This gives a odds ratio of:  $Odds\ ratio = e^{-0.361} \approx 0.697$ , meaning that if a fire occurs 1 degree further east, it is about 30 pct. less likely (odds-wise) to be catastrophic.

Our logistic regression model is interpretable and regularized, but performance could likely be improved with non-linear classifiers such as Random Forests and Boosting models.

## 3.2 Random Forest

### 3.2.1 Introduction to the Model

As our second model, we use Random Forest because it is a powerful and flexible ensemble method that performs well on classification tasks. It handles non-linear relationships, is robust to noise, and performs better than a single decision tree by averaging multiple trees to reduce variance.

Random Forest furthermore improves over bagged trees by adding a simple but effective twist: at each split, it only considers a random subset of predictors rather than all of them. This helps to decorrelate the trees, making the average prediction more stable and reliable, which can be very helpful, especially when predictors are correlated.

### 3.2.2 Hyperparameter tuning

To avoid the high computational cost of an exhaustive GridSearchCV, we first run a RandomizedSearchCV to explore a wide range of parameter combinations efficiently. This allows us to quickly find a good neighborhood of hyperparameters where performance is high.

RandomizedSearchCV randomly selects a number of parameter combinations (e.g., 20 in our case), trains and evaluates a model for each using 5-fold cross-validation, and returns the best-performing set. This rough tuning step helps narrow the parameter space before fine-tuning with GridSearchCV.

The parameters and ranges used in the RandomizedSearchCV can be found in Table 4.

For the Random Forest model, we also use `class_weight="balanced"` to ensure that both rare and frequent classes are weighted equally during model training. Again, we focus on the recall score of the catastrophic fires, as our primary goal is to correctly identify as many catastrophic cases as possible.

| Parameter         | Values Tested                       | Best Value |
|-------------------|-------------------------------------|------------|
| n_estimators      | Random integers between 100 and 300 | 120        |
| max_depth         | None, 10, 20, 30                    | 10         |
| max_features      | 'sqrt', 'log2'                      | 'log2'     |
| min_samples_split | Random integers between 2 and 10    | 5          |
| min_samples_leaf  | Random integers between 1 and 5     | 4          |

**Table 4:** Hyperparameter distributions used in RandomizedSearchCV

Increasing the number of iterations broadens the search space, and using more CV folds helps reduce the risk of overfitting. However, both also increase computation time, creating a trade-off between performance and time. Therefore, we chose 20 as our number of iterations. The same tradeoff is present for the number of trees. More trees generally reduce the variance and improve stability but many trees will increase the computing time. We chose to use between 100 and 300 trees, as that range offers a good balance of accuracy vs. compute time.

The depth of the tree can affect the performance of the tree, since allowing the tree to grow very deep can lead to overfitting. But since we are estimating a Random Forest, it is much less prone to overfitting, which is why we chose to also include max\_depth = none. However, we have quite unbalanced data which could potentially lead to overfitting, and in this case, limiting max\_depth can still improve generalization, and is thus worth tuning through cross-validation.

After identifying the best parameters from RandomizedSearchCV, which can be found in Table 4, we used them to define a narrower parameter grid for final tuning with GridSearchCV.

### 3.2.3 Final tuning/model

To fine-tune the model, we refined our search around the best parameters from our RandomizedSearchCV, allowing us to define our final parameter grid, which can be found in Table 5.

| Hyperparameter    | Values Tested  | Best Values |
|-------------------|----------------|-------------|
| n_estimators      | [90, 120, 150] | 90          |
| max_depth         | [10, 20, 30]   | 10          |
| max_features      | ['log2']       | 'log2'      |
| min_samples_split | [4, 5, 6]      | 4           |
| min_samples_leaf  | [3, 4, 5]      | 3           |

**Table 5:** Refined Hyperparameter Grid for GridSearchCV (Random Forest)

We again used class\_weight='balanced' to address class imbalance, scoring = recall score of the catastrophic fires, cv=5 for 5-fold cross-validation and GridSearchCV with n\_jobs=-1 to parallelize model fitting. As with logistic regression, we use the default threshold of 0.5 for classification. Alternative thresholds are explored later in the model comparison to evaluate their impact on recall.

With the parameter grid found in Table 5, we are able to do a focused optimization, using the most promising parameters, which will improve generalization while keeping compute time at

a reasonable level.

### 3.2.4 Model evaluation, interpretation and conclusion

| Metric / Feature           | Overall                 | Catastrophic (Class 1) | Other (Class 0) |
|----------------------------|-------------------------|------------------------|-----------------|
| Test Accuracy              | 0.735                   |                        |                 |
| OOB Score (Train Accuracy) | 0.747                   |                        |                 |
| Reduced Train Accuracy     | 0.727                   |                        |                 |
| Mean CV Accuracy (5-fold)  | 0.725                   |                        |                 |
| Recall                     |                         | 0.54                   | 0.84            |
| Precision                  |                         | 0.64                   | 0.77            |
| F1 Score                   |                         | 0.58                   | 0.81            |
| <b>Top 3 Predictors</b>    | <b>Importance Score</b> |                        |                 |
| Remoteness                 | 0.217                   |                        |                 |
| Longitude                  | 0.183                   |                        |                 |
| Latitude                   | 0.102                   |                        |                 |

**Table 6:** Summary of Random Forest Results (Catastrophic = 1)

From the results shown in Table 6, we see no strong evidence of overfitting.

The OOB score of 75.7 pct and cross-validation accuracy of 73 pct. are very close to the test accuracy of 73.5 pct. This consistency across training and validation metrics suggests that the model generalizes well.

The model is performing well for the majority class non-catastrophic, with a high recall score of 84 pct. and precision score of 77 pct. However, for the minority class catastrophic, precision is 64 pct, while the recall score is 54 pct, meaning that only 54 pct. of actual catastrophic fires are detected.

The most important predictors are 'Remoteness', 'Longitude' and 'Latitude', indicating that location helps to determine whether a fire is catastrophic or not, which we also saw earlier in the descriptive analysis part, from the strong correlation between these variables and the fire size. However, the score of the predictors is not really interpretable as the ones from the Logistic regression, and the scores only indicate that the predictors were frequently used in decision splits and contributes to prediction accuracy. However, it does not tell anything about whether a change in the predictor increases or decreases the risk of a catastrophic fire occurring.

This result clearly reflects the class imbalance in the data, making the model more biased towards the majority class, which makes detecting catastrophic fires harder.

Although Random Forests offer feature importance, their internal structure is more complex than simpler models like logistic regression, making causal interpretation harder.

## **3.3 Gradient Boosting**

### **3.3.1 Introduction to the Model**

Gradient Boosting models are among the most versatile and powerful machine learning methods available. They are well-suited for both regression and classification tasks. Unlike Random Forests, which rely on an ensemble of independent decision trees, Gradient Boosting Models build its trees sequentially. Each tree is a weak learner, typically a shallow decision tree, that attempts to correct the errors made by the previous ensemble. This characteristic, step-wise improvement is the defining feature of Gradient Boosting.

Each weak learner in the sequence has high bias and low variance, but as more learners are added, the model progressively reduces its bias, leading to a accurate final predictor. This method of building complexity through successive error correction allows the model to achieve high predictive performance.

Compared to Random Forests, Gradient Boosting does not rely on randomness to increase complexity and diversity in its model. It gradually constructs a complex and expressive model, which makes it more sensitive to overfitting and potentially more precise when optimized effectively.

### **3.3.2 Model Selection and Hyperparameter Tuning**

Increased performance in machine learning often coincides with higher computational requirements. This is particularly true for the traditional gradient boosting classifiers. However, alternative boosting algorithms such as Hist Gradient Boosting Classifier and Extreme Gradient Boosting (XGBoost) offer significantly faster training times. Both algorithms leverage histogram-based optimization, enabling faster computational efficiency and scalability.

In our case, the classification task involved a notable class imbalance between catastrophic and non-catastrophic fires. XGBoost provided advanced tools for handling imbalanced data and offered extensive hyperparameter tuning capabilities. For these reasons, XGBoost was selected as the most appropriate model for our analysis. Similar to the approach for Random Forests we made use of the two step Grid Search. First, we ran a RandomizedSearch CV to find a good neighborhood of hyperparameters where performance is high. Due to the lower computational constraints with XGBoost it was possible to run a higher number of iterations (100) making the randomizedsearchCV effective in finding strong hyperparameters. As in the Random Forest model, the randomized search is also trained using 5-fold cross-validation.

| Parameter        | Values Tested                           | Best Values |
|------------------|-----------------------------------------|-------------|
| n_estimators     | Randomized integers between 100 and 300 | 112         |
| max_depth        | [4, 5, 6, 8, 10]                        | 4           |
| learning_rate    | [0.05, 0.1]                             | 0.10        |
| gamma            | uniform (4.5, 3)                        | 6.19        |
| colsample_bytree | [0.5, 0.75]                             | 0.75        |
| min_child_weight | [4, 5, 6]                               | 6           |
| scale_pos_weight | [1.8, 2.0, 2.2]                         | 2.2         |

**Table 7:** Hyperparameter distributions used in RandomizedSearchCV

To handle the class imbalance between catastrophic and non-catastrophic fires, we adjusted the `scale_pos_weight` parameter in XGBoost. This parameter increases the penalty for misclassifying the minority class, which in this case, is the catastrophic fires. In a sense, it allows us to tune the model’s tendency to predict catastrophic or non-catastrophic fires during training, improving the recall rate. Since one third of the fires are considered to be catastrophic, the appropriate value for this parameter will lie around 2, reflecting the inverse ratio of the class frequencies.

The `max_depth` parameter controls the maximum depth of each individual decision tree in the ensemble. In contrast to Random Forest, Gradient Boosting models often perform better with shallower trees. This is due to the sequential nature of the model. The trees correct the residuals of the previous ones, so each tree doesn’t need to learn the full structure. As a result, shallow trees help to avoid overfitting, especially in boosting, where errors are decreasing across iterations. Our hyperparameter tuning confirmed that a tree depth between 3 and 10 is most effective, with an optimal depth of 3–5 in our case.

In addition to these core parameters, we also tuned `colsample_bytree`, `min_child_weight`, and `gamma` to further control the model’s complexity and regularization.

`colsample_bytree` determines the fraction of features to be randomly sampled for each tree. Reducing this value can prevent overfitting by introducing diversity across trees, similar to bagging in Random Forests.

`min_child_weight` sets the minimum sum of instance weights required in a child node. Higher values make the model more conservative, avoiding splits that occur due to small sample sizes or noise, thus improving robustness.

`gamma` specifies the minimum loss reduction required to make a split. A higher gamma makes the algorithm more selective when creating new branches, which helps prevent over-complex trees. In practice we experienced that increasing gamma improved the recall of catastrophic fires significantly, but at some expense of accuracy. However, since the recall improved notably up until a certain threshold with minimal trade-off, we chose to prioritize this improvement.

Tuning these parameters allows fine control over the bias-variance tradeoff, and was essential to optimizing performance while maintaining model generalization.

For our analysis the correct classification of catastrophic fire is extremely valuable to us and thus more important than pure accuracy of the predictions. Thus this focus was included in

our model.

After identifying promising parameters from RandomizedSearch CV, we performed a more focused search with GridSearchSV to fine-tune the optimal hyperparameters. Parameters not included in the grid were fixed at their optimal value determined by the randomized search. We used scoring=recall and cv=5 for 5-fold cross-validation, resulting in the optimal hyperparameters shown in Table 8. This two-step tuning process allowed us to refine the model more precisely, improving generalization and overall performance.

| Parameter        | Values Tested      | Best Values |
|------------------|--------------------|-------------|
| n_estimators     | 112                | 112         |
| max_depth        | [3, 4, 5]          | 3           |
| learning_rate    | [0.08, 0.10, 0.12] | 0.08        |
| gamma            | [5.89, 6.19, 6.49] | 5.89        |
| colsample_bytree | 0.75               | 0.75        |
| min_child_weight | 6                  | 6           |
| scale_pos_weight | 2.2                | 2.2         |

**Table 8:** Optimal Hyperparameters after GridSeachSV

### 3.3.3 Model evaluation, interpretation and conclusion

| Metric / Feature                 | Overall | Catastrophic (Class 1) | Other (Class 0) |
|----------------------------------|---------|------------------------|-----------------|
| <b>Test Accuracy</b>             | 0.68    |                        |                 |
| <b>Reduced Train Accuracy</b>    | 0.69    |                        |                 |
| <b>Mean CV Accuracy (5-fold)</b> | 0.67    |                        |                 |
| <b>Recall</b>                    |         | 0.70                   | 0.68            |
| <b>Precision</b>                 |         | 0.53                   | 0.81            |
| <b>F1 Score</b>                  |         | 0.60                   | 0.74            |

**Table 9:** Summary of Model Evaluation (Catastrophic = 1)

The objective of the model was to accurately predict whether a fire would be catastrophic. The boosting model achieved a recall of 70 pct., correctly identifying 1803 of the actual catastrophic fires. Thus the boosting model is outperforming the Random Forsest. However, missclassifying 30 pct. of catastrophic fires remains a significant limitation.

The improved recall rate for catastrophic fire comes at the expense of wrongfully classifying non-catastrophic fires as catastrophic. The recall for non-catastrophic fires is 68 pct., meaning that 32 pct., or 1,562, of these fires are mistakenly flagged as catastrophic.

Overall, the model's accuracy stands at 68 pct., closely aligned with the cross-validation accuracy of 67 pct. This consistency between training and validation performance suggests that the model generalizes well and does not suffer from significant overfitting.

Overall, the key consideration remains what type of error is more critical in a real-world scenario. Overclassifying fires as catastrophic or failing to identify catastrophic fires. In this model,

nearly half of the predicted catastrophic fires are false positives, potentially leading to misallocation of valuable firefighting resources. At the same time, 774 catastrophic fires go undetected. From this perspective, the model's performance remains insufficient.

## **3.4 Neural Network**

### **3.4.1 Introduction to the Model**

A feedforward neural network is an extremely flexible kind of machine learning model. Already with a single hidden layer it can approximate any continuous function on a compact domain (see Universal Approximation Theorem). As the possibilities of model design are basically endless they are very well suited for almost any regression or classification task. One of the major drawbacks of neural networks compared to other, simpler models like decision trees is the low interpretability. For this task we implemented the model using the PyTorch library.

### **3.4.2 Network Design and Hyperparameter Tuning**

The approach to choosing the right network design and hyperparameters was less methodic compared to the other models. We decided on a basic, general structure for the model first, and then fine-tuned certain parameters manually by trial-and-error on the validation set. While the Skorch library allows for GridSearchCV-like compatibility of PyTorch models in principal, we leave this option for further research. Note that this should certainly increase model performance. Instead of using k-fold cross validation we split the 80% train set by 80/20 again to achieve a total split of 64/16/20% of train/validation/test.

We decided on a basic structure of two hidden layers with ReLU activation and 64 and 32 units per layer, respectively. As we are dealing with a binary classification task, we used binary cross entropy as the loss function, from which the derivatives are calculated for backpropagation. The positive class was weighted by class ratio to tackle class imbalance. Since the model was trained in batches of size 128 we applied batch normalization after the second hidden layer to stabilize training and speed up convergence. To avoid overfitting, the probability of neuron dropout has been set to 0.5 for the second hidden layer.

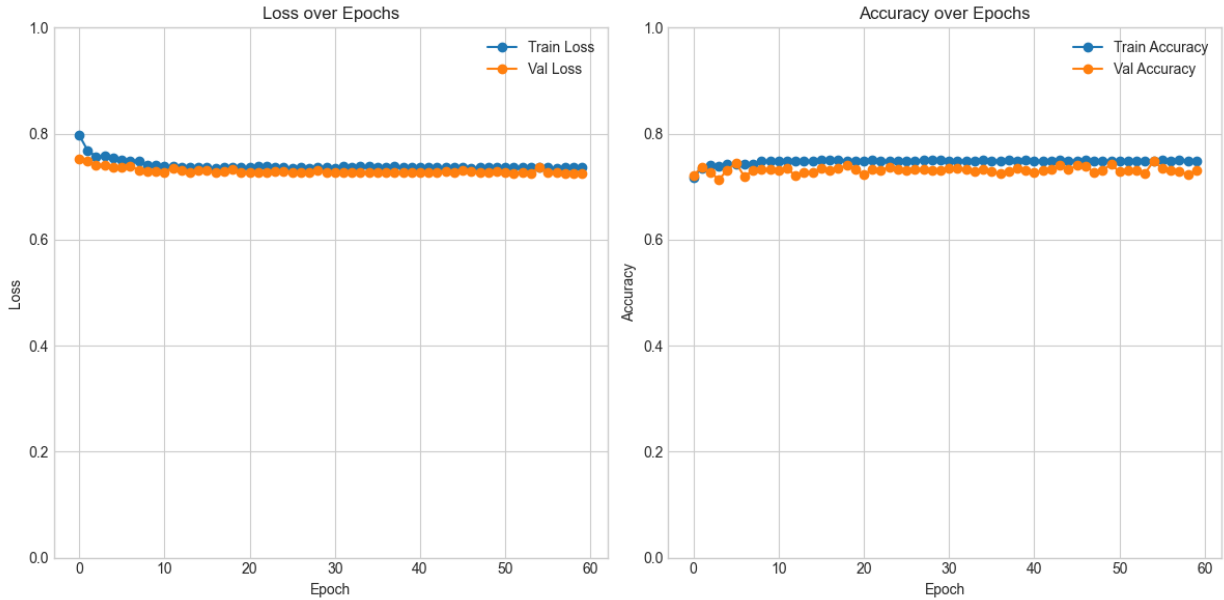
The parameters were trained over 60 epochs using the Adam optimizer (Adaptive Moment Estimation) with learning rate 0.01 and weight decay of 0.0001. They are initialized by default in PyTorch using the Kaiming Uniform. Additionally, we used a learning rate scheduler that multiplied the current learning rate by 0.1 if positive recall did not increase over three consecutive epochs. The minimum learning rate for the scheduler is 0.00001.

We tested a variety of combinations of layer sizes between 12-124 units per layer, dropout from 0 to 0.5, leaky and normal ReLU, and a few different batch sizes in terms of network design. Further, lower starting learning rates, higher weight decay and another optimizer (Stochastic Gradient Descent) were tested before deciding on the described hyperparameters and structure.

| Type                  | Description    |                     |
|-----------------------|----------------|---------------------|
| Input                 | Input Features | 24                  |
| Fully Connected (fc1) | Linear Layer   | $in = 24, out = 64$ |
| Activation            | ReLU           | -                   |
| Fully Connected (fc2) | Linear Layer   | $in = 64, out = 32$ |
| BatchNorm             | BatchNorm1d    | -                   |
| Activation            | ReLU           | -                   |
| Dropout               | Dropout Layer  | $p = 0.5$           |
| Fully Connected (fc3) | Linear Layer   | $in = 32, out = 1$  |
| Output                | Raw Logits     | 1                   |

**Table 10:** Network Design.

### 3.4.3 Evaluation



**Figure 5:** Loss and Accuracy over Training Epochs.

Especially the dropout was helpful in combating overfitting. Using the default classification threshold of 0.5, the train and validation accuracy seem to mostly converge after around ten epochs to around 0.72 to 0.75 with minimal overfitting. While the accuracy does not seem to be too bad the positive validation recall value of 0.47 of the final epoch is worrying. Only 47% of catastrophic fires were correctly identified as such. 48% on the test set do not look much better.

| Class            | Precision | Recall | F1-score | Support |
|------------------|-----------|--------|----------|---------|
| Not Catastrophic | 0.76      | 0.86   | 0.81     | 4865    |
| Catastrophic     | 0.65      | 0.48   | 0.55     | 2577    |
| <b>Accuracy</b>  |           |        | 0.73     | 7442    |
| Macro avg        | 0.70      | 0.67   | 0.68     | 7442    |
| Weighted avg     | 0.72      | 0.73   | 0.72     | 7442    |

**Table 11:** Classification Report on Test Set.

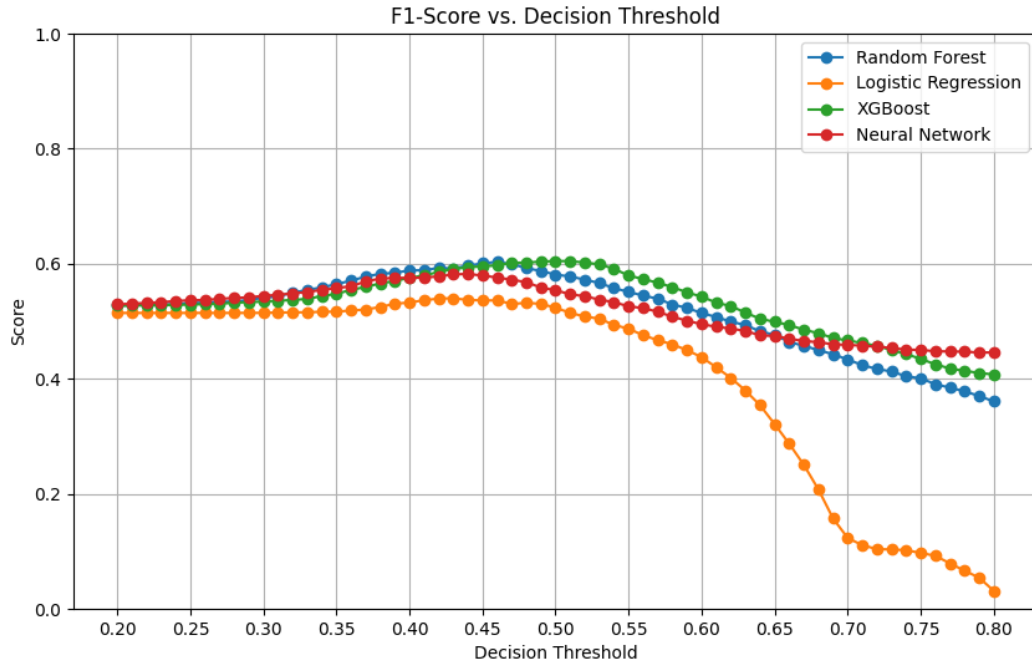
### 3.5 Comparison of the Models

In the table below you can find seven models evaluated on the test set, sorted by F1 score. Apart from the XGBoost model every model could be improved in terms of positive F1 score by setting a custom decision threshold below 0.5. Here we chose the threshold that maximizes the respective F1 score of the positive class (catastrophic). For example, if the neural network outputs a probability of 0.44 for an observation to be catastrophic, we classify it as catastrophic already. Thus we gain sensitivity towards the positive class. Please note that the optimal decision threshold **must** be decided on during the training process. Here we simply want to show how it affects the results and thus simply choose it from test set performance.

| Model                                   | Accuracy | Recall | Precision | F1   | F1 (weighted) |
|-----------------------------------------|----------|--------|-----------|------|---------------|
| XGBoost                                 | 0.68     | 0.70   | 0.53      | 0.60 | 0.69          |
| Random Forest with threshold 0.46       | 0.70     | 0.67   | 0.55      | 0.60 | 0.70          |
| Ensemble Model (Majority Vote)          | 0.69     | 0.66   | 0.54      | 0.60 | 0.70          |
| Neural Network with threshold 0.44      | 0.66     | 0.69   | 0.50      | 0.58 | 0.66          |
| Random Forest                           | 0.73     | 0.54   | 0.63      | 0.58 | 0.73          |
| Logistic Regression with threshold 0.43 | 0.52     | 0.82   | 0.40      | 0.54 | 0.51          |
| Logistic Regression                     | 0.66     | 0.54   | 0.51      | 0.52 | 0.66          |

**Table 12:** Test Set Model performance sorted by F1 score.

Especially considering the already optimal default decision threshold, we find the XGBoost model to perform the best in our task. With a custom threshold the random forest does come pretty close to it though. We are quite certain that with a more methodic approach to network design and tuning the neural network could still be improved to the level of XGBoost. Logistic regression could potentially be improved by feature engineering, such as creating feature interaction variables regarding weather for example.



**Figure 6:** F1-Score vs Decision Threshold across Models

Using the optimal thresholds we finally created an ensemble model of our four models which lies in the middle performance wise. This indicates that rather than capturing different information in the data, the models vary more in their capability of extracting information at all.

## 4 Conclusion and Outlook

How useful are our models for helping with wildfire triage in real life? And how could we improve the models?

First of all, we can argue that the models are at least a lot better than guessing. But institutions fighting wildfires do not guess either. Further they have a lot more information that is not included in our dataset. Most importantly, we did not have access to any weather forecasts upon discovery of a fire. If a fire is discovered but heavy rain is expected for the following week, the containment efforts can obviously be much lower compared to an extended dry spell. In our opinion, including forecasts should increase the model performance quite a bit.

But with intelligent feature engineering, like time to previous fire (as fires do not burn well in fully burned areas), model performance could also be raised slightly.

In conclusion, in its current state, our algorithms are not of much use to firefighting institutions, but promising directions of improvement are visible.

## 5 Sources

- Dataset: <https://www.kaggle.com/datasets/capcloudcoder/us-wildfire-data-plus-other-attributes/code>
- <https://medium.com/@rithpansanga/logistic-regression-and-regularization-avoiding-overfitting-and-improving-generalization-e9afdcddd09d>
- <https://medium.com/@piyushkashyap045/a-comprehensive-guide-to-gradient-boosting-and-regression-in-machine-learning-step-by-step-faa17bd0e2c>
- <https://medium.com/@amit25173/xgboost-vs-gradient-boosting-d7797d1ab751::text=XGBoost>
- <https://www.pexels.com/photo/forest-on-fire-51951/>